Assumption University of Thailand

Vincent Mary School of Engineering, Science and Technology

CSX3009 – Algorithm Design

Semester 2/2025

Algorithm Design Term Project Report

Submitted to

Dr. Kwankamol Nongpong

Submitted by

Thit Lwin Win Thant (ID: 6540122)

Kaung Khant Lin (ID: 6540131)

Submission Date: March 12, 2026

Table of Contents

Table of Contents **0**
1. Problem Description **1**
1.1 Overview 1
1.2 The Optimization Constraint 1
1.3 Input and Output Specification 2
1.3.1 Input Format 2
1.3.2 Output Format 2
2. The Logic of the Algorithm **3**
2.1 The Failure of Local Optimization (Greedy Approach) 3
2.2 The Core Mechanic: Per-Type Minimums 3
2.3 The Interval DP Recurrence Relation 4
2.4 Bottom-Up Execution 4
3. Walkthrough of a Sample Case **5**
3.1 Stage 1: Manual Greedy Trace (10 Operations) 5
3.2 Stage 2: Naive Recursion Walkthrough 5
3.3 Stage 3: Top-Down DP (Memoization) 6
3.4 Stage 4: Interval DP Mathematical Walkthrough (8 Operations) 6
4. Time and Space Complexity Analysis **7**
4.1 Test cases results 7
4.2 Time and Space Complexity Analysis 8
4.2.1 Brute-Force (Naive Recursion) 8
4.2.2 Memoization (Top-Down DP) 8
4.2.3 Interval DP (Bottom-Up) 9
5. Appendix **10**

1. Problem Description

1.1 Overview

We are tasked with optimizing a prescribed weightlifting training routine. The routine consists of a sequence of exercises that must be completed in a strict order. Each exercise requires a specific multiset of weight plates to be present on the machine.

The machine operates strictly as a *Last-In-First-Out (LIFO)* stack, similar to a vertical loading pin.

- A single operation consists of either adding (pushing) one weight plate to the top of the stack, or removing (popping) one weight plate from the top of the stack.
- The stack must begin completely empty before the first exercise, and it must be returned to a completely empty state after the final exercise.
- The objective is to find a stacking strategy that minimizes the total number of operations (pushes + pops) required to complete the entire routine.

1.2 The Optimization Constraint

If the machine required us to load the weights in a specific order, the sequence of operations would be deterministic and trivial. However, the problem introduces one critical flexibility rule: **the required weights for any individual exercise can be loaded onto the stack in any order.**

Because we control the internal arrangement of the stack for each exercise, we have an optimization opportunity. If multiple consecutive exercises require the same specific weights, we can strategically place those shared weights at the absolute bottom of the stack. This allows us to leave them untouched while transitioning between exercises, thereby eliminating redundant push and pop operations.

1.3 Input and Output Specification

1.3.1 Input Format

The program must process multiple independent test cases.

- ***T*:** The first line provides the number of test cases.
- ***E* and *W*:** Each test case begins with the number of exercises (E, up to 100) and the number of distinct weight types (W, up to 100).
- **The Requirement Matrix (*X*):** The next E lines contain W integers each. This forms an $E \times W$ matrix where the cell $X\{i,j\}$ represents exactly how many weight plates of type j are required to be on the stack for exercise i.

1.3.2 Output Format

For each test case, the program must output a single line in the format: Case #x: y Where x is the 1-based test case number, and y is the absolute minimum number of operations required to complete that specific sequence of exercises

2. The Logic of the Algorithm

2.1 The Failure of Local Optimization (Greedy Approach)

To understand the algorithm, we must first establish why a naive Greedy approach fails. A Greedy algorithm optimizes strictly local transitions; for example, looking only at the shared weights between Exercise 1 and Exercise 2.

This fails because this problem contains temporal dependencies. A weight placed at the absolute bottom of the stack during Exercise 1 might not be needed for Exercise 2, but could be strictly required for Exercise 3. A local optimization will pop that weight during Exercise 2, forcing redundant push and pop operations later. To find the optimal solution, we must foresee which weights form the "global base" for a sequence of exercises.

2.2 The Core Mechanic: Per-Type Minimums

Our objective is to identify a "global base stack" that remains completely untouched across a contiguous interval of exercises [*i, j*].

Mathematically, the maximum number of reusable plates for an interval is the per-type minimum across that interval.

For each weight type *t*, the shared amount is: min(*X{i,t}, X{i+1,t}, ... , X{j,t}*).

If any single exercise within the interval requires fewer copies of a specific weight type, that exercise acts as a bottleneck, limiting how many plates can safely reside at the absolute bottom of the stack without forcing a teardown.

2.3 The Interval DP Recurrence Relation

Because an exhaustive brute-force search evaluates the exact same overlapping sub-intervals repeatedly (resulting in an exponential O(4^E / E^{3/2}) time complexity), we use Interval Dynamic Programming (DP). Interval DP systematically evaluates every possible binary split of a sequence, caches the optimal cost of small intervals, and merges them to build the full sequence.

The recurrence relation to merge two adjacent intervals (Left and Right) is:

Total Cost = Cost(Left) + Cost(Right) - Duplicated Base

- **Base Case:** For a single exercise [i, i], the cost is 2 * (total weights in exercise i). Every required plate must be pushed once and popped once.
- **The Subtraction:** Both the Left and Right subproblems independently calculate the cost of pushing and popping their own shared bases from an empty stack. When we merge them, the global base shared across the entire merged interval should only be built (pushed) once and destroyed (popped) once. Adding Left + Right double-counts these operations. We must subtract 2 * (size of global shared base) to eliminate the duplicated push/pop cost.

2.4 Bottom-Up Execution

To guarantee that subproblems are solved before they are referenced, the DP table is populated bottom-up by strictly increasing interval length (from length 1 up to E). For every interval, the algorithm tests every possible split point k, applying the recurrence relation to find the absolute minimum cost.

3. Walkthrough of a Sample Case

To demonstrate the evolutionary logic of our algorithm, we trace a 3-exercise workout using three weight types (Red, Blue, Green) through our development stages.

Input Matrix (test3.in):

- **Exercise 1:** `[1, 1, 1]` (Needs 1 Red, 1 Blue, 1 Green)
- **Exercise 2:** `[1, 1, 0]` (Needs 1 Red, 1 Blue)
- **Exercise 3:** `[0, 1, 1]` (Needs 1 Blue, 1 Green)

3.1 Stage 1: Manual Greedy Trace (10 Operations)

A Greedy algorithm optimizes only the immediate transition from Exercise 1 to 2. It identifies shared Red and Blue weights, and arbitrarily stacks them as `[Red, Blue, Green]` (bottom to top).

1. **Ex 1:** Push all 3. Stack is `[Red, Blue, Green]`. (Cost: 3 ops).
2. **Transition to Ex 2:** Pop Green. Stack is `[Red, Blue]`. (Cost: 1 op).
3. **Transition to Ex 3 (Needs Blue, Green):** The required Blue is trapped under the Red. The stack must be torn down. Pop Blue, Pop Red, Push Blue, Push Green. Stack is `[Blue, Green]`. (Cost: 4 ops).
4. **End:** Empty the machine. Pop Green, Pop Blue. (Cost: 2 ops).

Total operations: 10. (Greedy fails due to the trapped weight).

3.2 Stage 2: Naive Recursion Walkthrough

To avoid the Greedy trap, the naive recursive approach explores every possible binary partition. For `[Ex 1, Ex 2, Ex 3]`, it tests two splits:

- **Split A:** `([Ex 1])` merged with `([Ex 2, Ex 3])`
- **Split B:** `([Ex 1, Ex 2])` merged with `([Ex 3])`

It calculates the push/pop combinations for both branches and returns the minimum. While this finds the optimal 8 operations, if E scales to 100, exploring every branch triggers a Catalan-bound combinatorial explosion O(4^E / E^{3/2}), making execution impossible.

3.3 Stage 3: Top-Down DP (Memoization)

To prevent the recursive explosion, we introduce a dictionary (a "Notepad") to cache computed intervals. If Split A evaluates the sub-interval `[Ex 2, Ex 3]`, it saves the result. If a different branch later asks for `[Ex 2, Ex 3]`, the algorithm instantly retrieves the cached value instead of re-calculating it. This collapses the massive tree into a Directed Acyclic Graph (DAG).

3.4 Stage 4: Interval DP Mathematical Walkthrough (8 Operations)

The final Bottom-Up Interval DP removes recursion entirely. It populates a 2D matrix by strictly increasing interval lengths, guaranteeing optimal substructure.

Step 1: Length 1 Intervals (Base Cases)

- `dp[1, 1]` (Ex 1): 2 × 3 weights = 6 ops.
- `dp[2, 2]` (Ex 2): 2 × 2 weights = 4 ops.
- `DP[3, 3]` (Ex 3): 2 × 2 weights = 4 ops.

Step 2: Length 2 Intervals

- `dp[1, 2]`: Ex 1 and Ex 2 share Red and Blue (2 weights). Savings = 4.
  Cost: dp[1,1] + dp[2,2] - 4 → 6 + 4 - 4 = 6 ops.
- `dp[2, 3]`: Ex 2 and Ex 3 share Blue (1 weight). Savings = 2.
  Cost: dp[2,2] + dp[3,3] - 2 → 4 + 4 - 2 = 6 ops.

Step 3: Length 3 Interval (The Full Sequence)

The global shared base across all three exercises is just Blue (1 weight). Savings = 2. The algorithm evaluates all splits to find `DP[1, 3]`:

- **Split after Ex 1:** dp[1,1] + dp[2,3] - Savings → 6 + 6 - 2 = 10 ops.
- **Split after Ex 2:** dp[1,2] + dp[3,3] - Savings → 6 + 4 - 2 = 8 ops.

The DP table outputs 8 operations. Physically, this represents anchoring the global shared base (Blue) at the absolute bottom of the stack (`[Blue, Red, Green]`), avoiding the forced teardown seen in the Greedy trace.

4. Time and Space Complexity Analysis

4.1 Test cases results

The three algorithms were evaluated using several test_cases. The small *test1.in* dataset verified correctness. The others were generated with increasing *E*, *W*, and *X_i* values to test scalability.

Test case	**Constraint (E, W, Xi)**	**Naive runtime (ms)**	**Memoization runtime (ms)**	**Interval DP runtime (ms)**
test1.in	E ⩽ 10, W ⩽ 3, Xi ⩽ 3	0.060	0.055	0.062
large_test1.in	E = 20, W = 20, Xi ⩽ 20	-	16.777	3.367
large_test2.in	E = 40, W = 40, Xi ⩽ 40	-	270.479	32.690
large_test3.in	E = 60, W = 60, Xi ⩽ 60	-	1454.823	126.545
large_test4.in	E = 80, W = 80, Xi ⩽ 80	-	4784.407	328.367
large_test5.in	E = 100, W = 100, Xi ⩽ 100	-	11859.221	667.641

4.2 Time and Space Complexity Analysis

Algorithm	**Time Complexity**	**Space Complexity**
Naive Recursion	$O(4^E / E^{\frac{3}{2}})$	O(E)
Memoization (Top-Down DP)	$O(E^3 \times W)$	$O(E^2)$
Interval DP (Bottom-Up)	$O(E^3 + E^2 W)$	$O(E^2)$

4.2.1 Brute-Force (Naive Recursion)

- **Time Complexity:** $O(4^E / E^{\frac{3}{2}})$ The naive algorithm explores every possible binary partition of the sequence. Because branches recombine combinatorially, the number of operations scales according to the Catalan numbers. This exponential explosion is physically evident in our results (Table 4.1), where the algorithm fails to complete (times out) for any test case where E ⩽ 20.
- **Space Complexity:** O(E) Despite its catastrophic runtime, it is highly memory efficient. It only requires space for the recursive Call Stack, which reaches a maximum depth equal to the number of exercises, E.

4.2.2 Memoization (Top-Down DP)

- **Time Complexity:** $O(E^3 \times W)$ Memoization collapses the exponential tree into a Directed Acyclic Graph (DAG) by caching the results of overlapping sub-intervals. There are $O(E^2)$ unique sub-intervals. For each interval, the algorithm tests O(E) split points. However, within each state evaluation, it recalculates the shared weights using a linear scan, adding an $O(E \times W)$ overhead. This brings the total time to polynomial $O(E^3 \times W)$. As seen in Table 4.1, this allows the algorithm to survive E=100, but it takes nearly 12 seconds, pushing dangerously close to standard competitive programming time limits.
- **Space Complexity:** $O(E^2)$ The space complexity increases quadratically because we must store the computed result for all $O(E^2)$ possible sub-intervals in a dictionary, alongside the O(E) recursive Call Stack.

4.2.3 Interval DP (Bottom-Up)

- **Time Complexity:** $O(E^3 + E^2 W)$ The final Interval DP approach optimizes the Memoization logic by extracting the shared-weight calculation. By precomputing all shared bases into a 2D array bottom-up $O(E^2 W)$, the lookup cost during the state transitions becomes strictly O(1). Testing O(E) split points across $O(E^2)$ intervals therefore takes a strict $O(E^3)$ time. The empirical data validates this mathematical optimization: by removing both the O(W) inner-loop bottleneck and the overhead of Python's recursive Call Stack, the runtime for E=100 drops from 11.8 seconds (Memoization) to just 0.6 seconds (Interval DP).
- **Space Complexity:** $O(E^2)$ The space requirement remains quadratic to store the E × E DP table and the precomputed E × E shared weights matrix. However, because it is strictly iterative, it completely eliminates the risk of Stack Overflow errors present in the Top-Down approach.

5. Appendix

test3.in

```
1
3 3
1 1 1
1 1 0
0 1 1
```